ECE 597 Capstone Project: Multi-Stage IoT Network Intrusion Detection Using Unsupervised and Supervised Learning

Ardeshir Shojaeinasab, Riham AlTawy
Department of Electrical and Computer Engineering

Summer 2026

1 Project Overview

You will build a two-stage intrusion detection system (IDS) for IoT networks using the CIC IoT-DIAD 2024 dataset. The first stage applies unsupervised learning on packet-level data to flag anomalies (an anomaly-based IDS). The second stage applies supervised learning on flow-level features to re-check those flags, reducing false positives while keeping overall detection strong (a signature-based IDS).

2 Learning Objectives

By the end of the project you should be able to:

- Handle imbalanced cybersecurity datasets through targeted sampling.
- Apply unsupervised methods to detect anomalies in network traffic.
- Build supervised classifiers on flow-level features.
- Compare different machine learning approaches on the same security task.
- Reason about the trade-off between false positives and false negatives in IDS design.
- Evaluate your system with metrics that matter for security applications.

3 Dataset

You will use the **CIC IoT-DIAD 2024 dataset**, which captures IoT network traffic under several attack scenarios. Both packet-level and flow-level features are provided. The five attack categories you will work with are:

- DDoS-HTTP_Flood (distributed denial of service, HTTP variant)
- DoS-HTTP_Flood (denial of service, HTTP variant)
- DNS Spoofing

- Cross-Site Scripting (XSS)
- Brute Force

A note on attack selection. Flow-level data for the TCP Flood variants of DDoS and DoS is not available in this release of the dataset. Use the HTTP Flood variants for both the DoS and DDoS categories throughout the project.

3.1 Dataset Access

- **Download link**: `http://cicresearch.ca/IOTDataset/CIC%20IoT-IDAD%20Dataset%202024/`
- **Dataset information**: `https://www.unb.ca/cic/datasets/iot-diad-2024.html`
- Download *both* the packet-level and flow-level files. You will need both.

4 Project Requirements

4.1 Phase 1: Data Preparation and Sampling

4.1.1 Task 1.1: Random Dataset Generation Function

Write a Python function that samples the packet-level dataset randomly according to these specifications:

- **Benign traffic**: 200,000 rows (roughly 97–98% of the sample).
- **Attack traffic**: 4,000–6,200 rows (roughly 2–3% of the sample).
- Attack traffic should be randomly distributed across the five attack types listed above.
- Randomization must produce a different composition on each run. A configurable random seed is a good idea.
- Include basic validation and integrity checks.

4.1.2 Task 1.2: Data Preprocessing

Prepare the data so the downstream models have a fair chance:

- Handle missing values and outliers.
- Apply scaling or normalization where it helps.
- Use feature selection or dimensionality reduction if it improves results.
- Address any quality issues specific to network traffic data.
- Document every preprocessing choice with a short justification.

4.2 Phase 2: Unsupervised Learning for Initial Detection (Anomaly-Based IDS)

4.2.1 Task 2.1: Packet-Level Anomaly Detection

Build an unsupervised model that separates benign from non-benign packets.

- Any unsupervised method is fine. Autoencoders and k-means are reasonable starting points, and you can combine them: for example, train an autoencoder, take its reconstruction loss as a feature, then cluster the result with k-means so the clusters separate benign from non-benign groups. Use whatever you can justify.
- Do not use packet labels during training.
- Tune hyperparameters systematically where applicable.
- Generate initial alerts based on anomaly scores or cluster assignments.

4.2.2 Task 2.2: Alert Generation and Analysis

- Pick thresholds for the packet-level anomaly detector and justify them.
- Analyze the balance between false positives and false negatives.
- Report initial detection performance.

4.3 Phase 3: Supervised Learning for False Positive Reduction (Signature-Based IDS)

Every packet in the packet-level dataset has corresponding flow data in the flow-level dataset, identified by a flow ID of the form

`[source_IP]-[destination_IP]-[sender_port]-[receiver_port]`.

The flow-level dataset may contain multiple rows for a single flow, because flows longer than 2 minutes are automatically split into successive 2-minute segments that share the same flow ID. When you process flow data, group by flow ID and use the right aggregation per feature (mean, sum, or max, depending on what the feature means) to produce one unified record per flow. You will need to identify the flow-level rows that correspond to the packet-level dataset generated in Phase 2.

You should also generate a second random dataset directly from the flow-level data, using the same proportions (200,000 benign rows and 4,000–6,200 non-benign rows). This flow-level dataset is what you will use for the supervised stage.

Figures 1 and 2 show the correspondence between a packet-level row and the flow-level rows derived from it.

4.3.1 Task 3.1: Flow-Level Anomaly Detection (Optional but Recommended)

Optional enhancement. Apply the same anomaly-based approach you used in Phase 2 to the flow-level data.

- Run your unsupervised method on the flow-level dataset, using the flows that correspond to the data points you selected at the packet level.

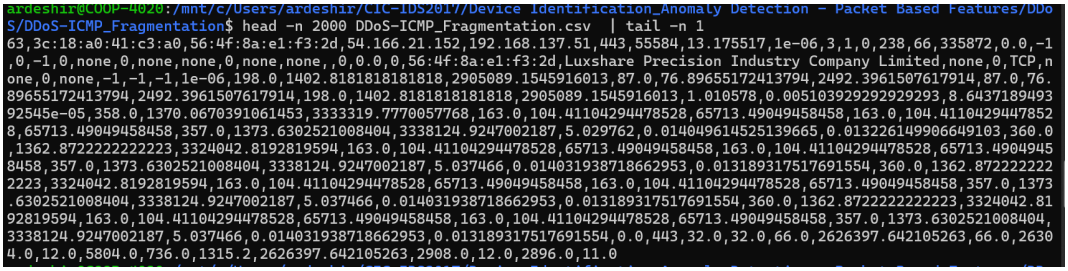

Figure 1: Packet-level data row example.

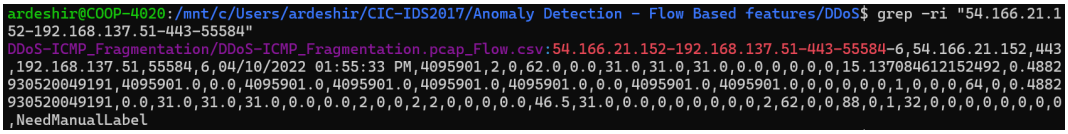

Figure 2: Corresponding flow-level data.

- Use what you learn from this to inform the supervised stage that follows.
- Compare packet-level and flow-level unsupervised performance. Test the hypothesis that flow-based features perform better in the unsupervised setting and report what you find.

4.3.2 Task 3.2: Flow-Based Feature Engineering

- For each packet flagged as anomalous in Phase 2, locate the corresponding flow-level data.
- Generate a new random dataset over flow-based features using the same sampling function from Task 1.1.
- Remember the flow segmentation behaviour described above: group rows by flow ID and apply per-feature aggregations to produce one record per flow.
- If you completed Task 3.1, use its findings to guide feature selection and engineering.

4.3.3 Task 3.3: Signature-Based IDS Implementation

Build a supervised model that re-evaluates the alerts from Phase 2.

- Use any model you can defend: Graph Neural Networks, Random Forest, SVM, deep neural networks, XGBoost, ensembles, or something else. Pick what fits your data.
- If you completed Task 3.1, you can incorporate features derived from flow-level anomaly detection. Run the supervised model with and without those features so the contribution is clear.
- Use proper train/validation/test splits.
- Use the trained model to re-classify the packets initially flagged in Phase 2.
- Aim to reduce false positives while preserving detection accuracy.

5 Evaluation and Metrics

5.1 Required Performance Metrics

5.1.1 Phase 2 (Packet-Level Unsupervised Detection)

- Precision, recall, and F1-score for overall detection.
- Per-attack detection rate for each of the five attack types.
- False Positive Rate (FPR) and False Negative Rate (FNR).
- AUC-ROC.
- Confusion matrix.
- A short analysis of how you handled the imbalance in the data.

5.1.2 Phase 3 (Flow-Level Supervised Refinement)

- Percentage reduction in false positives compared to Phase 2.
- Overall system accuracy after the two stages run together.
- Class-level evaluation and performance report.
- Precision and recall for the combined system.
- Computational overhead.
- Time-complexity comparison between the two phases.

5.1.3 Comparative Analysis

- How does your two-stage system compare to single-stage approaches?
- How does performance vary across attack types?
- Are your improvements statistically significant?
- Which attack types can be detected well using packet-level detection alone, and why do you think that is?
- How do DDoS and DoS look in your unsupervised clusters at the packet level (and at the flow level, if you implemented Task 3.1)?

6 Guidelines

6.1 Novelty and Innovation

You are encouraged to:

- Try unusual combinations of anomaly-based and signature-based approaches. A familiar technique can still count as novel here if you have a good reason for using it and the results back you up.

- Engineer features carefully. This does not need to be exotic. A simple transformation that clearly improves performance is worth a lot.
- Look into ensembles, or graph-based methods such as GATs for richer feature representations. These are options, not requirements. A simpler approach that works well is fine.
- Document every approach you tried, including the ones that did not work. Track results on the validation set so the progression from your first attempt to your final model is visible.
- Think about the hybrid architecture as a whole, not just the parts.
- **Note**: I evaluate projects on novelty, performance, and the quality of the report.

7 Git Workflow and Development Process

You must maintain a Git repository for this project from day one. The commit history is part of how I evaluate your work, so treat it accordingly.

- **Commit small and often.** A series of meaningful commits with clear messages tells a much better story than a single "final submission" dump at the end. I will look at the timeline.
- **Make individual contributions visible.** Each team member's work should show up in the log under their own identity. If two people work on the same file, use co-authored commits or separate commits. Do not share accounts or have one person push everyone's work.
- **Push regularly.** Do not keep weeks of work on a single laptop.
- **Include the repository link** (and ideally read access for the instructor and TA) with your final submission.

A reasonable history shows steady progress over the project window. A wall of activity in the last 48 hours before the deadline is not what I am looking for, and it will affect the grade.

8 Use of AI Tools

AI chatbots such as ChatGPT, Claude, Gemini, and similar tools are **allowed** for assistance.

AI coding agents that complete tasks end-to-end with minimal human involvement, including Cursor in agent mode, Claude Code in autonomous mode, Devin, Copilot Workspace agent mode, and similar tools, are **not allowed**. If your submission shows signs of having been produced by an agent rather than by you, you will fail the project. Signs include: generic structure with no real engagement with the specifics of the dataset, no incremental commit history, code or analysis that you cannot explain in the demo, or boilerplate that does not match what the project actually asked for.

If you use AI chatbots, the following rules apply:

- **Understand everything you submit.** I or a TA may ask you to walk through any part of your code or report in detail. I expect a real answer, not a recap of what the chatbot told you.
- **Track your usage as you go.** Keep notes on where you used AI assistance and what you used it for.

- **Cite specific uses in the report.** "I used ChatGPT" is not enough. For each instance, note which tool, what task (for example: debugging a specific function, drafting a paragraph of the introduction, explaining a concept you were stuck on), and how you verified or modified the output.
- **Acknowledge the type of usage honestly.** Brainstorming, debugging, code generation for a specific function, writing assistance, and so on are all different uses, and each one should be labelled for what it was.

The demo exists for a reason: I want to hear you explain what you built, in your own words.

9 Deliverables

9.1 Code Submission

- A complete, runnable Python implementation.
- Clear documentation and inline comments.
- A video (no longer than 15 minutes) showing the project from execution through results and visualization, with clear narration.
- A README with setup and execution instructions.
- A `requirements.txt` file listing all dependencies.
- Your Git repository link (or the `.git` directory itself).

9.2 Written Report

The report should include the following sections.

9.2.1 1. Introduction and Methodology

- Problem statement and an overview of your approach.

9.2.2 2. Implementation Details

- Preprocessing steps and the reasoning behind them.
- Model architectures and how you chose your hyperparameters.
- Training and optimization procedures.

9.2.3 3. Results and Analysis

- Performance metrics with clear visualizations.
- Statistical analysis of the results.
- Comparison tables showing how performance changes across phases.
- ROC curves, precision-recall curves, and confusion matrices.
- A performance breakdown by attack type (DDoS-HTTP_Flood, DoS-HTTP_Flood, DNS Spoofing, XSS, Brute Force).

9.2.4 4. Discussion and Insights

Address the following:

- **Flow-based advantages**: How does using flow-level information improve detection accuracy? Show evidence.
- **Flow length analysis**: Does longer flow duration or higher packet count correlate with better classification accuracy? Support this with data.
- **Attack-type analysis**: Which of the five attack types benefits most from the two-stage approach, and why?
- **False positive reduction**: Quantify the reduction in Phase 3 and explain the mechanism behind it.

9.2.5 5. Conclusion and Future Work

- Summary of your key findings and contributions.
- Limitations of the current approach.
- Suggestions for future improvements.

9.3 AI Usage Statement

Include a section in the report listing every AI tool you used, what you used it for, and how you verified the output. See the "Use of AI Tools" section above for the level of detail expected.

10 Evaluation Criteria

Component	Weight	Description
Technical Implementation	30%	Code quality, correctness, and completeness
Performance Results	25%	Accuracy, improvement metrics, evaluation thoroughness
Novelty and Innovation	20%	Creative approaches and original contributions
Report Quality	15%	Writing clarity, visualizations, and analysis depth
Discussion and Insights	10%	Quality of conclusions and depth of analysis

Table 1: Project evaluation rubric.

The Git history and the demo both feed into the Technical Implementation and Report Quality scores. A submission that cannot be reproduced from the repository, or that the team cannot explain in the demo, will not score well regardless of the numerical results.

11 Important Notes

- **Academic integrity**: All implementations must be your own original work. Cite any referenced methods or code properly.
- **Dataset validation**: I will independently validate reported results using separate train/validation/test splits to ensure fair evaluation.

- **Reproducibility**: All results must be reproducible from your code and documentation.
- **Late submission**: Standard course late-submission policies apply.
- **External references**: Any external research papers, open-source projects, code repositories, or methodologies used in your implementation must be cited with proper academic references.

Good luck. This is a chance to apply real machine learning to a real cybersecurity problem, and to come out of it with skills you can actually point to.